LAPORAN PRAKTIKUM

Struktur Data

MODUL 6

DOUBLY LINKED LIST (BAGIAN PERTAMA)



Disusun oleh:

Dharma Chandra Viriya

109082500052

S1IF-13-02

PROGRAM STUDI S1 INFORMATIKA

FAKULTAS INFORMATIKA

TELKOM UNIVERSITY PURWOKERTO

TAHUN 2026

Link Repo Github Modul 6 : https://github.com/iamCelott/praktikum-struktur-data-smt3-modul-6

LATIHAN SOAL MODUL

1. **Soal 1**

Source Code - Doublylist.h

```
#ifndef DOUBLYLIST_H
#define DOUBLYLIST_H

#include <iostream>
#include <string>
using namespace std;

struct infotype {
  string nopol;
  string warna;
  int thnBuat;
};

typedef struct ElmList *address;

struct ElmList {
  infotype info;
  address next;
  address prev;
};

struct List {
  address First;
  address Last;
};

void CreateList(List &L);
address alokasi(infotype x);
void dealokasi(address &P);
void printInfo(List L);
void insertLast(List &L, address P);

address findElem(List L, infotype x);

void deleteFirst(List &L, address &P);
void deleteLast(List &L, address &P);
void deleteAfter(List &L, address Prec, address &P);

#endif
```

Source Code - Doublylist.cpp

```
#include "Doublylist.h"

void CreateList(List &L) {
  L.First = nullptr;
  L.Last = nullptr;
}

address alokasi(infotype x) {
  address P = new ElmList;

  P->info = x;
  P->next = nullptr;
  P->prev = nullptr;

  return P;
}

void dealokasi(address &P) {
  delete P;
  P = nullptr;
}

void printInfo(List L) {
  address P = L.Last;

  while (P != nullptr) {
    cout << "no polisi : " << P->info.nopol << endl;
    cout << "warna     : " << P->info.warna << endl;
    cout << "tahun     : " << P->info.thnBuat << endl;

    P = P->prev;
  }
}

void insertLast(List &L, address P) {
  if (P == nullptr) {
    return;
  }

  P->next = nullptr;
  P->prev = nullptr;

  if (L.First == nullptr) {
    L.First = P;
    L.Last = P;
  } else {
    P->prev = L.Last;
```

```
      L.Last->next = P;
      L.Last = P;
    }
  }

  address findElem(List L, infotype x) {
    address P = L.First;

    while (P != nullptr) {
      if (P->info.nopol == x.nopol) {
        return P;
      }

      P = P->next;
    }

    return nullptr;
  }

  void deleteFirst(List &L, address &P) {
    if (L.First == nullptr) {
      P = nullptr;
      return;
    }

    P = L.First;

    if (L.First == L.Last) {
      L.First = nullptr;
      L.Last = nullptr;
    } else {
      L.First = L.First->next;
      L.First->prev = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
  }

  void deleteLast(List &L, address &P) {
    if (L.Last == nullptr) {
      P = nullptr;
      return;
    }

    P = L.Last;

    if (L.First == L.Last) {
```

```
      L.First = nullptr;
      L.Last = nullptr;
    } else {
      L.Last = L.Last->prev;
      L.Last->next = nullptr;
    }

    P->next = nullptr;
    P->prev = nullptr;
  }

  void deleteAfter(List &L, address Prec, address &P) {
    if (Prec == nullptr || Prec->next == nullptr) {
      P = nullptr;
      return;
    }

    P = Prec->next;
    Prec->next = P->next;

    if (P == L.Last) {
      L.Last = Prec;
    } else {
      P->next->prev = Prec;
    }

    P->next = nullptr;
    P->prev = nullptr;
  }
```

Source Code - main.cpp

```
  #include "Doublylist.h"
  #include <iostream>

  using namespace std;

  bool cekNopol(List L, string nopol) {
    infotype x;
    x.nopol = nopol;

    return findElem(L, x) != nullptr;
  }

  bool hapusKendaraan(List &L, string nopol) {
    infotype x;
    address P;
    address Prec;
```

```
    x.nopol = nopol;

    P = findElem(L, x);

    if (P == nullptr) {
      return false;
    }

    if (P == L.First) {
      deleteFirst(L, P);
    } else if (P == L.Last) {
      deleteLast(L, P);
    } else {
      Prec = P->prev;
      deleteAfter(L, Prec, P);
    }

    dealokasi(P);

    return true;
  }

  int main() {
    List L;
    infotype kendaraan;
    address P;
    address hasil;

    string cari;
    string hapus;

    CreateList(L);

    for (int i = 1; i <= 4; i++) {
      cout << "masukkan nomor polisi: ";
      cin >> kendaraan.nopol;

      cout << "masukkan warna kendaraan: ";
      cin >> kendaraan.warna;

      cout << "masukkan tahun kendaraan: ";
      cin >> kendaraan.thnBuat;

      cout << endl;

      if (cekNopol(L, kendaraan.nopol)) {
        cout << "nomor polisi sudah terdaftar" << endl;
```

```
      } else {
        P = alokasi(kendaraan);
        insertLast(L, P);
      }

      cout << endl;
    }

    cout << "DATA LIST 1" << endl << endl;
    printInfo(L);

    cout << "\nMasukkan Nomor Polisi yang dicari : ";
    cin >> cari;

    kendaraan.nopol = cari;
    hasil = findElem(L, kendaraan);

    if (hasil != nullptr) {
      cout << "\nNomor Polisi : " << hasil->info.nopol << endl;
      cout << "Warna        : " << hasil->info.warna << endl;
      cout << "Tahun        : " << hasil->info.thnBuat << endl;
    } else {
      cout << "Data kendaraan tidak ditemukan" << endl;
    }

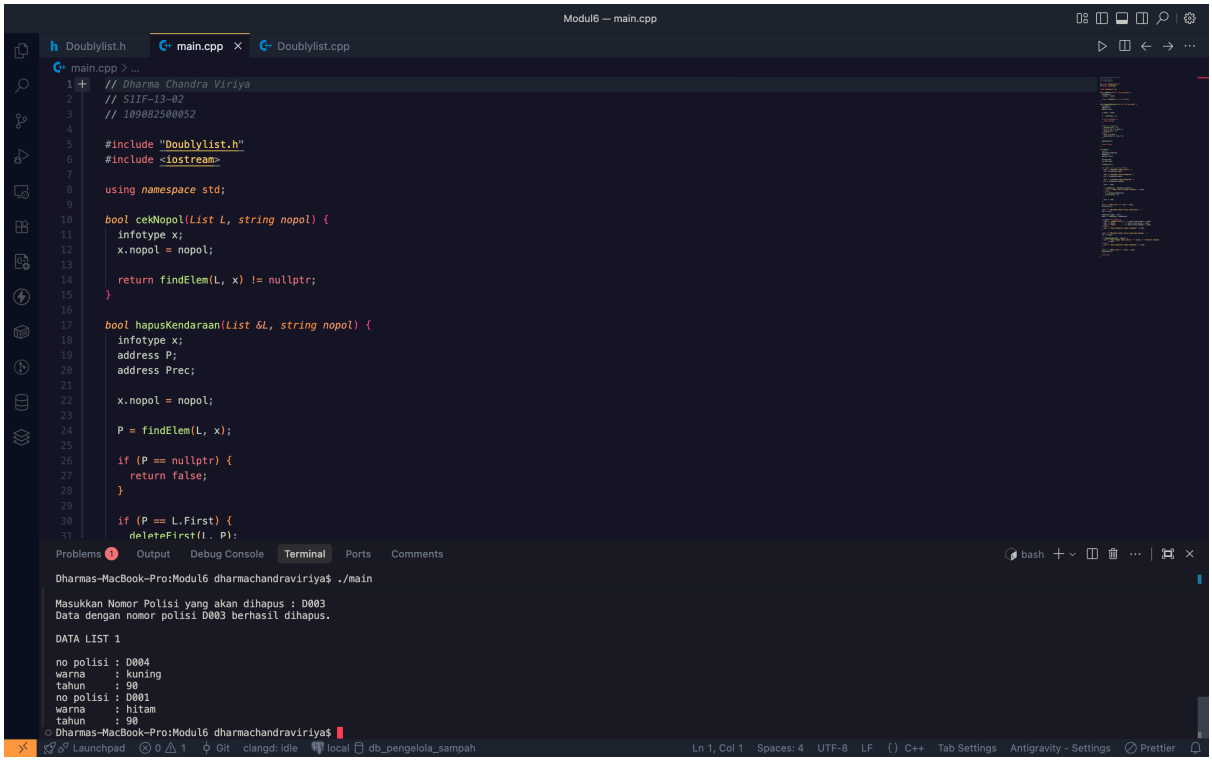
    cout << "\nMasukkan Nomor Polisi yang akan dihapus : ";
    cin >> hapus;

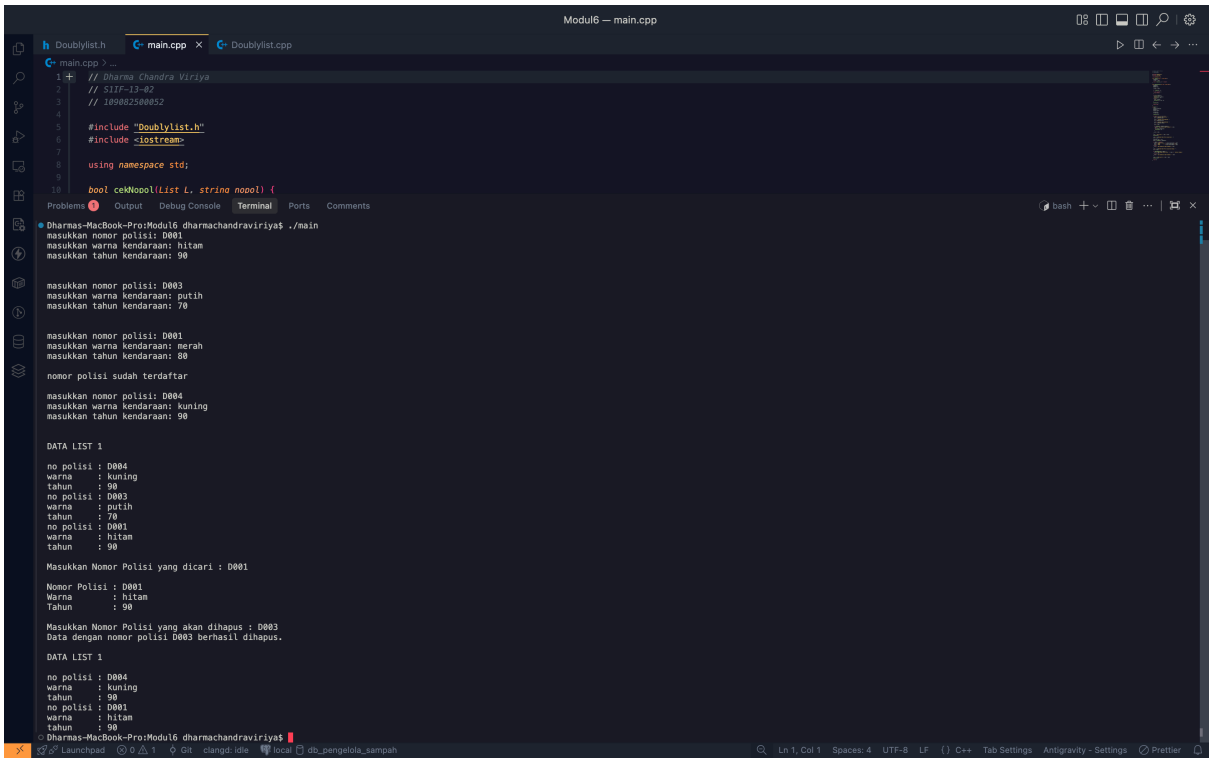
    if (hapusKendaraan(L, hapus)) {
      cout << "Data dengan nomor polisi " << hapus << "
  berhasil dihapus."
           << endl;
    } else {
      cout << "Data kendaraan tidak ditemukan." << endl;
    }

    cout << "\nDATA LIST 1" << endl << endl;
    printInfo(L);

    return 0;
  }
```

Screenshoot program





Deskripsi program

Program ini merupakan implementasi Abstract Data Type (ADT) Doubly Linked List menggunakan bahasa pemrograman C++ yang dirancang untuk mengelola data kendaraan secara dinamis. Setiap data kendaraan memiliki tiga atribut, yaitu nomor polisi, warna kendaraan, dan tahun pembuatan. Struktur Doubly Linked List digunakan karena setiap elemen memiliki dua pointer, yaitu next dan prev, yang saling terhubung untuk menunjukkan elemen berikutnya dan sebelumnya. Dengan struktur tersebut, proses penelusuran data dapat dilakukan dari awal menuju akhir maupun sebaliknya. Implementasi program dibagi menjadi tiga file, yaitu Doublylist.h sebagai tempat deklarasi struktur data dan prototype fungsi, Doublylist.cpp sebagai tempat implementasi prosedur dan fungsi ADT, serta main.cpp sebagai program utama untuk menjalankan dan menguji seluruh fitur yang tersedia.

Fitur yang dikembangkan dalam program ini meliputi pembuatan list kosong menggunakan CreateList, alokasi memori untuk membuat elemen baru melalui alokasi, serta dealokasi untuk membebaskan memori ketika elemen dihapus. Penambahan data kendaraan dilakukan menggunakan insertLast, yaitu dengan menghubungkan elemen baru pada posisi paling akhir list melalui pointer next dan prev. Program juga dilengkapi fungsi findElem untuk mencari kendaraan berdasarkan nomor polisi, sehingga pengguna dapat menemukan informasi kendaraan tanpa harus menelusuri seluruh data secara manual. Selain itu, terdapat validasi untuk mencegah penggunaan nomor polisi yang sama agar tidak terjadi duplikasi data. Proses penghapusan elemen diimplementasikan melalui deleteFirst, deleteLast, dan deleteAfter yang masing-masing digunakan untuk menghapus elemen pertama, terakhir, dan elemen setelah posisi tertentu. Seluruh data kendaraan dapat ditampilkan menggunakan printInfo dengan memanfaatkan pointer prev, sehingga data ditampilkan dari elemen terakhir menuju elemen pertama. Melalui implementasi ini, program menerapkan konsep struktur data dinamis, pengelolaan memori, manipulasi pointer, serta operasi dasar Doubly Linked List dalam pengelolaan data kendaraan.